Lecture 7

- HTTP Features: Cookies and Web Caching
- HTTP/2 and HTTP/3

HTTP Client-Server Demo

- Running HTTP server “locally”
- Background 1:
  - IP address of the server: 127.0.0.1
  - This is the standard IPv4 “loopback address”, also known as **localhost**
- Background 2:
  - *“cURL”* is a command-line tool for sending requests to servers over the network.
  - it acts like a very simple web client. Instead of using Chrome or Firefox, we type a command such as:

```
curl --http1.1 -v http://127.0.0.1:8000/
```

  - curl sends an HTTP request to the server and prints the response in the terminal.

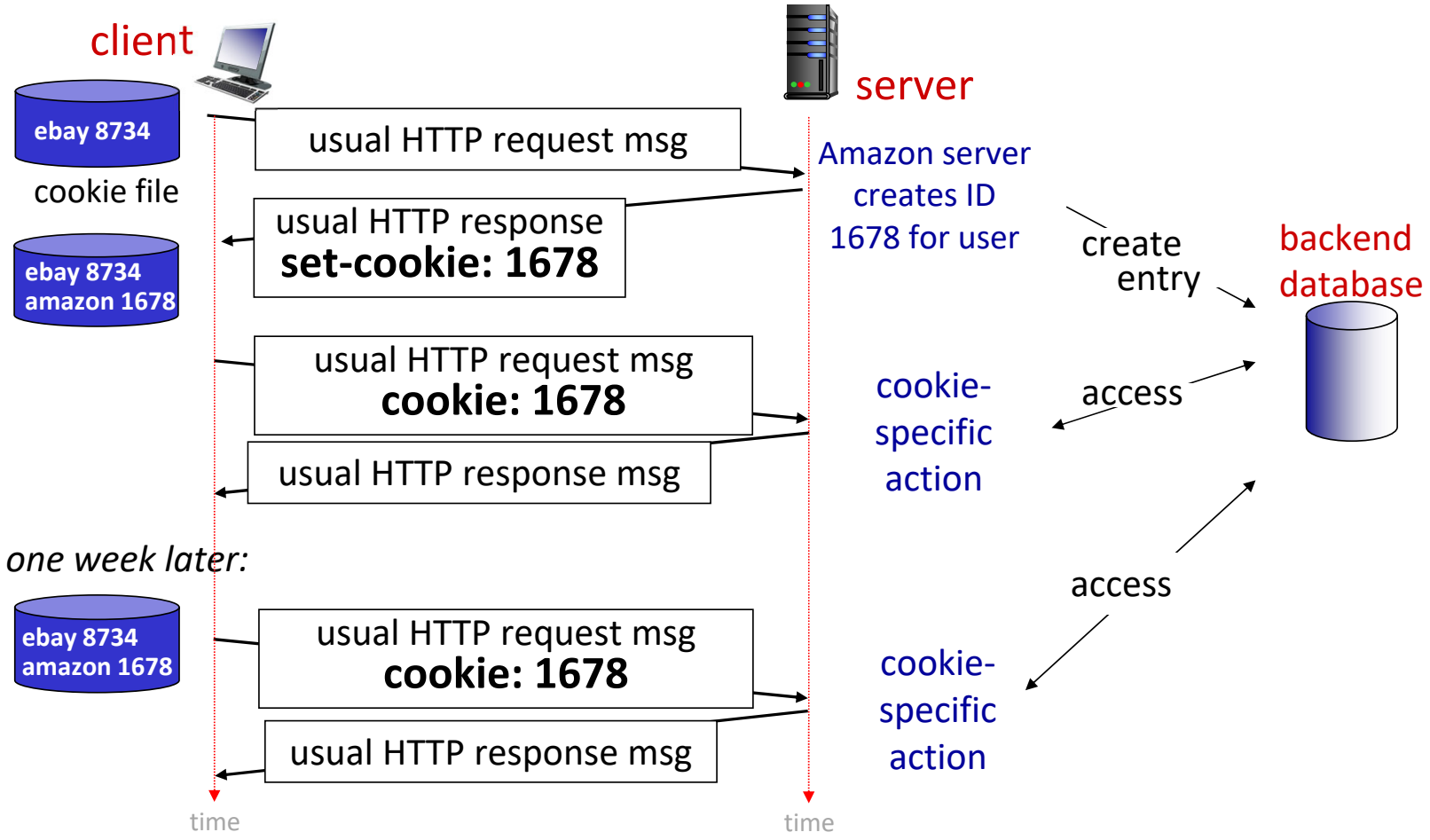
Maintaining user/server state: cookies

Web sites and client browser use *cookies* to maintain some state between transactions

four components:

1) cookie header line of HTTP *response* message
2) cookie header line in next HTTP *request* message
3) cookie file kept on user's host, managed by user's browser
4) back-end database at Web site

Maintaining user/server state: cookies



HTTP cookies: comments

What cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

Challenge: How to keep state:

- protocol endpoints: maintain state at sender/receiver over multiple transactions
- cookies: HTTP messages carry state

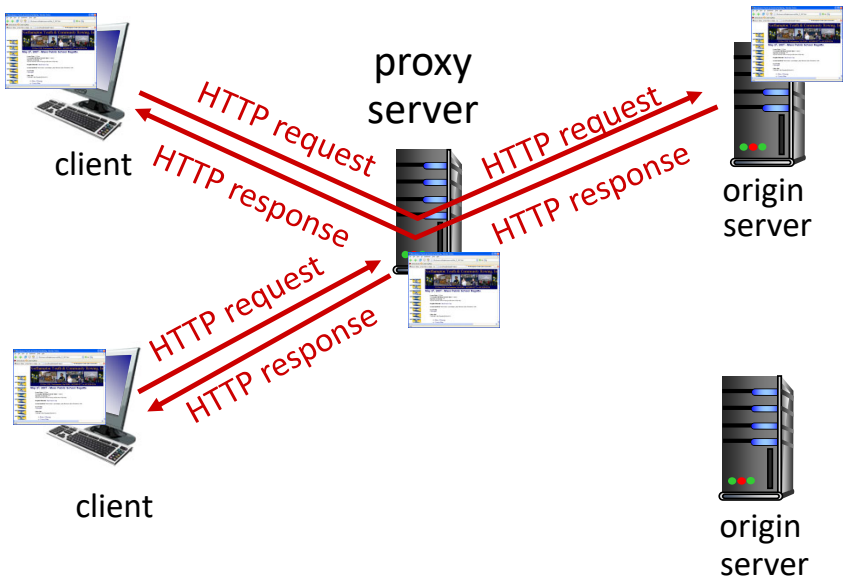
cookies and privacy:

- cookies permit sites to *learn* a lot about you on their site.
- ***Third party persistent cookies*** (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

Web caches (proxy servers)

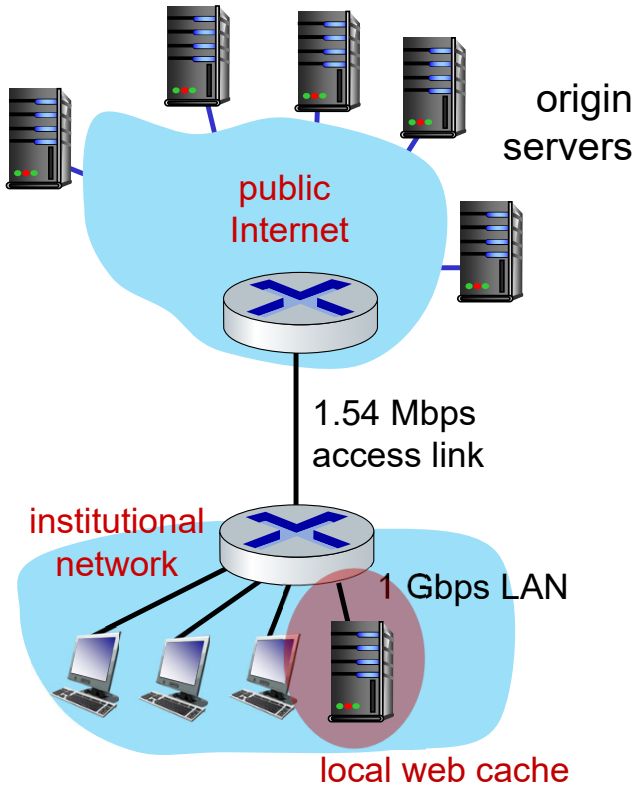
Cache is "close" to clients (e.g., in the same network)

- Browser sends all HTTP requests to cache
  - ***if object in cache***: cache returns object to client
  - ***else*** cache requests object from origin server, caches received object, then returns object to client



Installing a web cache

- Web cache acts as both client and server
  - server for original requesting client
  - client to origin server
- Typically, cache is installed by ISP (university, company, residential ISP)
- Smaller response time (cache “closer” to client)
- Decrease traffic to distant servers
  - link out of institutional/local ISP network often bottleneck



Conditional GET

Goal: don't send object if cache has up-to-date cached version

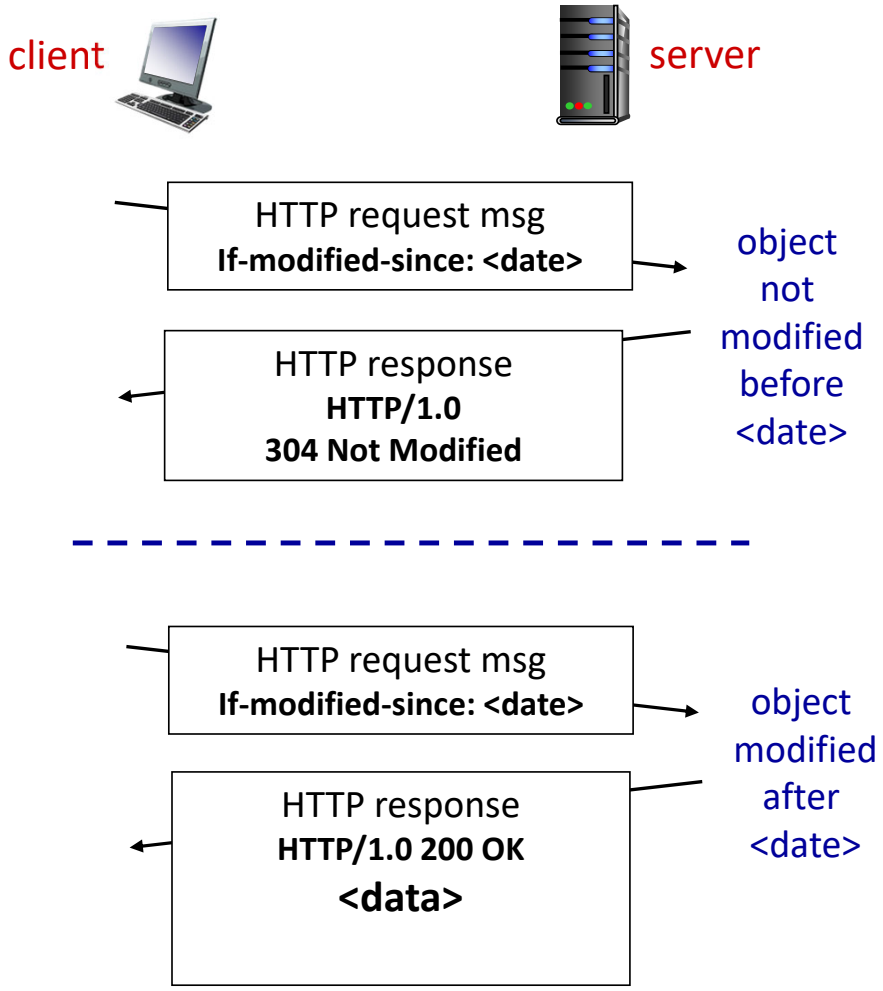
- no object transmission delay
- lower link utilization

- *cache:* specify date of cached copy in HTTP request

  If-modified-since: <date>

- *server:* response contains no object if cached copy is up-to-date:

  HTTP/1.0 304 Not Modified



HTTP/2

- HTTP server responds *in-order* (FCFS: first-come-first-served scheduling) to GET requests.
- With FCFS, small objects may have to wait for transmission (head-of-line (HOL) blocking) behind large object(s)

HTTP/2 was developed to decrease delay in multi-object HTTP requests!

HTTP/2

HTTP/2: [RFC 7540, 2015] increased flexibility at *server* in sending objects to client:

- Methods, status codes, most header fields unchanged from HTTP 1.1
- transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- *Push* unrequested objects to client
- Divide objects into frames, schedule frames to mitigate HOL blocking

HTTP/2

Internet Engineering Task Force (IETF)
Request for Comments: 7540
Category: Standards Track
ISSN: 2070-1721

M. Belshe
BitGo
R. Peon
Google, Inc
M. Thomson, Ed.
Mozilla

May 2015

Hypertext Transfer Protocol Version 2 (HTTP/2)

Abstract

This specification describes an optimized expression of the semantics of the Hypertext Transfer Protocol (HTTP), referred to as HTTP version 2 (HTTP/2). HTTP/2 enables a more efficient use of network resources and a reduced perception of latency by introducing header field compression and allowing multiple concurrent exchanges on the same connection. It also introduces unsolicited push of representations from servers to clients.

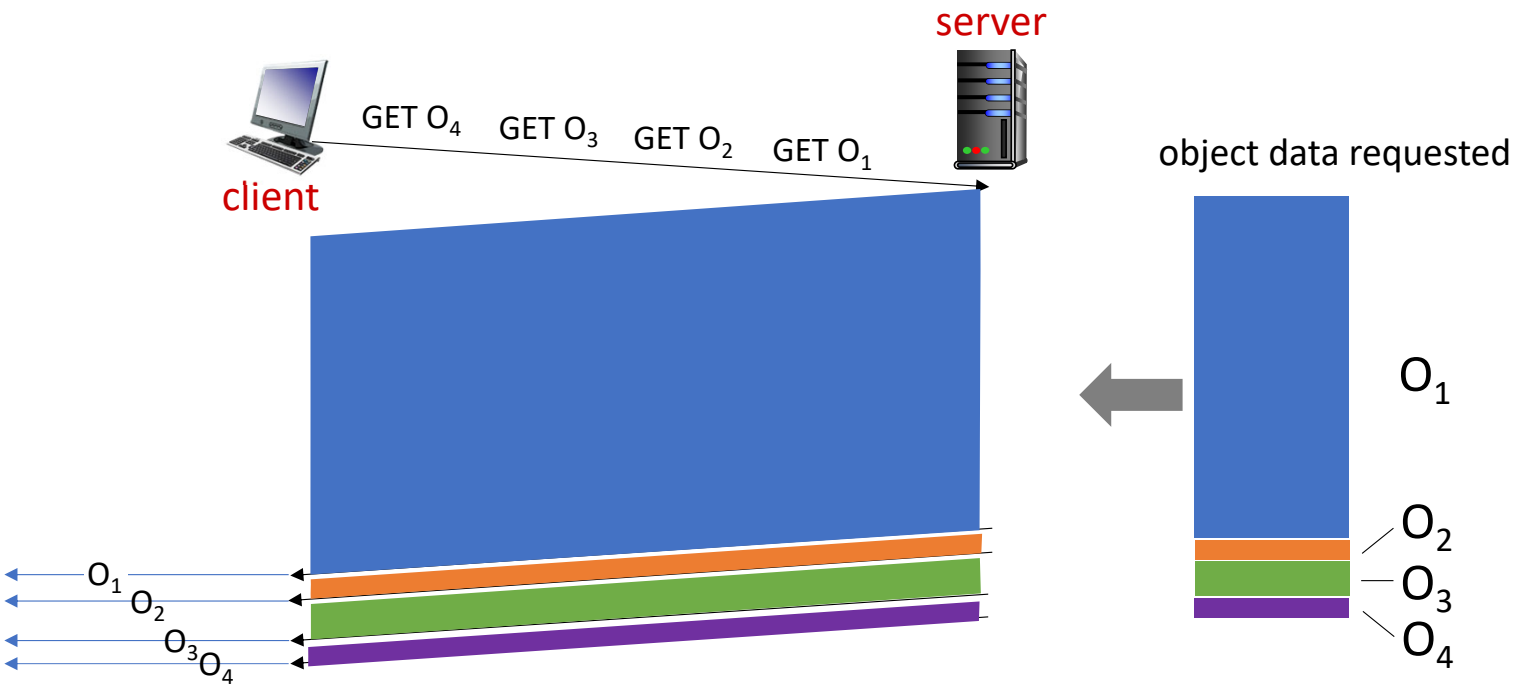
This specification is an alternative to, but does not obsolete, the HTTP/1.1 message syntax. HTTP's existing semantics remain unchanged.

Status of This Memo

This is an Internet Standards Track document.

HTTP/2: mitigating HOL blocking

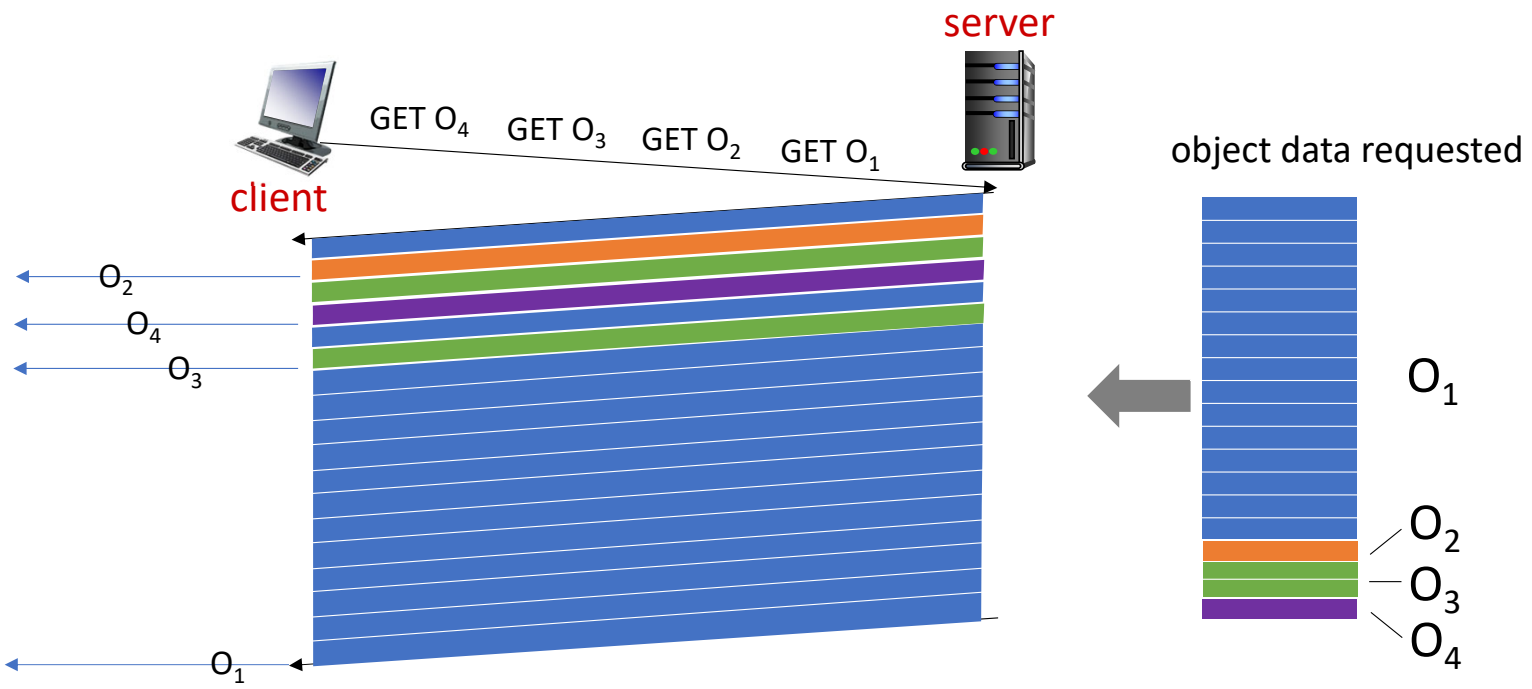
HTTP 1.1: client requests 1 large object (e.g., video file, and 3 smaller objects)



objects delivered in order requested: O_2, O_3, O_4 wait behind O_1

HTTP/2: mitigating HOL blocking

HTTP/2: objects divided into frames, frame transmission interleaved



O_2, O_3, O_4 delivered quickly, O_1 slightly delayed

HTTP/2 to HTTP/3

- HTTP/2 had no security over vanilla TCP connection
- HTTP/3: adds security , per object error- and congestion-control (more pipelining)
- HTTP/3 runs on top of QUIC transport protocol
- **QUIC**: Quick UDP Internet Connections

HTTP/2 to HTTP/3

Internet Engineering Task Force (IETF)
Request for Comments: 9114
Category: Standards Track
Published: June 2022
ISSN: 2070-1721

M. Bishop, Ed.
Akamai

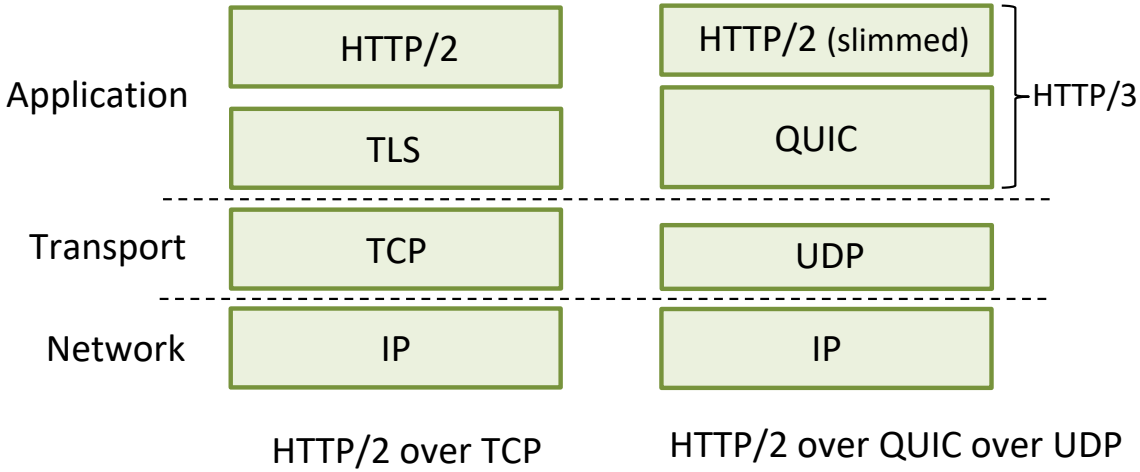
HTTP/3

Abstract

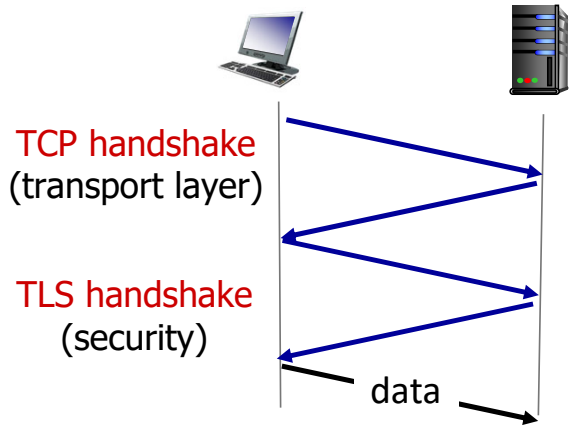
The QUIC transport protocol has several features that are desirable in a transport for HTTP, such as stream multiplexing, per-stream flow control, and low-latency connection establishment. This document describes a mapping of HTTP semantics over QUIC. This document also identifies HTTP/2 features that are subsumed by QUIC and describes how HTTP/2 extensions can be ported to HTTP/3.

QUIC: Quick UDP Internet Connections

- application-layer protocol, on top of UDP
  - increase performance of HTTP
  - deployed on many Google servers, apps (Chrome, mobile YouTube app)

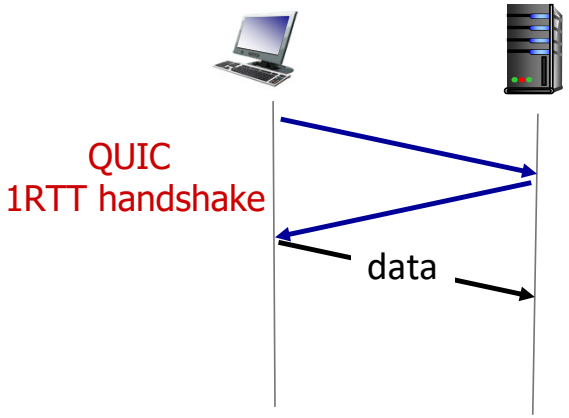


QUIC: Connection establishment



TCP (reliability, congestion control state) + TLS (authentication, crypto state)

- 2 serial handshakes



QUIC: reliability, congestion control, authentication, crypto state

- 1 handshake